

SOFTWARE ENGINEERING

Lab 4: Vibe Coding

Name : Inchara K Kuppal	SRN: PES1UG25CS818	Class: 5 'F' CSE
-------------------------	--------------------	------------------

Claude - Vibe coding tool chat history

<https://claude.ai/share/207cd5bb-5fc4-4dac-b066-dbaf19b4cead>

https://github.com/inchara13kuppal/62_scramble

You are my step-by-step lab coach and pair-programming partner. Guide me from start to finish. Teach me as we go: after each fix or feature, explain in 2-3 lines what the code does and why, so I learn it.

ASSIGNMENT CONTEXT

- Repo assigned to me: https://github.com/SETAPESU26/62_scramble (Python + Pygame Word Scramble game, "Word Scramble Repair Lab")
- Files: main.py, game/game_engine.py, game/text_box.py, README.md
- Objective: use an LLM (you) to fix the broken code and add features, prompting efficiently (aim for 3-4 prompt attempts in total), then push to MY OWN personal GitHub account.
- The work is individual. Do NOT raise a PR to the main repo (SETAPESU26).
- Time budget: ~60 minutes.

TASKS (do in this order, one at a time, testing after each)

1. Bug fix: in game_engine.submit_guess(), is_correct = (guess == self.scrambled_word) must compare against self.secret_word.
2. Hint system: a clickable HINT button beside SUBMIT. Each click reveals the first unrevealed letter of self.secret_word in its correct position (e.g. P _ _ _ _ for PYTHON) and deducts a small point penalty from self.score. Handle edge cases: all letters revealed, reset hints each round.
3. Countdown timer: a 20-second timer bar drawn in game_engine.render(). On expiry, reveal the secret word, show a "TIME'S UP!" warning, and auto-call self.next_round(). Reset the timer every round.
4. Letter tiles: render each scrambled letter as its own square tile; let the player click (or drag) tiles into a rearrangement rack to try different orders before submitting.

Expected behavior to preserve: correct guess = +1 point, success message, next word; wrong guess = warning, clear the input box, don't advance; empty submission = prompt with no penalty.

DELIVERABLES (all must be in a Lab-4/ folder in my personal GitHub repo)

- a. Video BEFORE changes (10 sec, showing the bug: typing the right word gets "WRONG GUESS!")
- b. Video AFTER changes (10 sec, showing the bug fixed + hint, timer, and tiles working)
- c. Updated code
- d. Chat history exported as PDF/doc (also keep the chat share link, since the repo README asks for it)

CONSTRAINTS

- My laptop has limited disk space: use git clone --depth 1, no virtual environment, install only pip install pygame, record short low-resolution clips (720p or lower), add a .gitignore for __pycache__/, and delete raw recordings after pushing. Don't suggest large installs (no OBS, no Docker).
- Keep all work in THIS single chat, because the chat history is graded.

HOW YOU SHOULD GUIDE ME

- FIRST, ask me two things: my OS (Windows/Mac/Linux) and my GitHub username. Then tailor every command to my OS.
- Give ONE small step at a time and wait for me to reply "done" (or paste an error) before the next step. Keep steps short.
- Cover these phases, in order, each with exact commands or clicks:
 1. Environment: check Python 3.10+, install pygame, open a terminal/VS Code, verify python main.py runs.
 2. Fork the repo to my account, clone it shallowly, open the folder in VS Code.
 3. STOP and REMIND me to record the 10-second BEFORE video (tell me exactly what to show and how to record with built-in tools). Don't continue until I confirm.
 4. Ask me to paste game_engine.py, text_box.py and main.py. Read them, then help fix Task 1 and explain why the bug happens.
 5. Tasks 2, 3, 4 one by one: give me the exact prompt I should send (or send it yourself), show only the changed functions and where they go, and tell me how to test each. If something breaks, ask for the traceback.
 6. STOP and REMIND me to record the 10-second AFTER video, and tell me what to show.
 7. Create the Lab-4/ folder, place the videos and code in it, and give the exact git commands

(add, commit, push to MY repo, no PR).

8. Tell me how to export this chat as PDF and where to put it, and how to copy the share link.

9. Run a final checklist against all deliverables and the README submission checklist before I submit.

- Keep track of which phase we're in. At the end of each phase, say "Phase X done, next: Y".

- If I ask something off-track, answer briefly and return to the current step.

- Be honest: if you're unsure about my code, ask me to paste it instead of guessing.

acknowledge and Start now with the two questions (OS and GitHub username).

i need very clear steps including opening vs code , setting github , creating folder, and so on

Let's get you full marks on this one. We'll go one small step at a time, and I'll tell you when each phase is done.

Phase 1 of 9: Environment setup

Before I give you any commands, I need two things from you so every step matches your machine:

1. Which operating system are you on? (Windows, Mac, or Linux)
2. What is your GitHub username? (the personal account you'll push to, not the SETAPESU26 org)

Reply with both and I'll give you Step 1.

win 11

inchara13kuppall

Read, added to, and updated memory

Windows 11 and inchara13kuppall, noted. Every command from here is for Windows.

Phase 1 of 9: Environment. Step 1 of 4: Check that Python is installed

1. Press the Windows key, type cmd, and press Enter. A black Command Prompt window opens.
2. Type this and press Enter:

`python --version`

What you should see: Python 3.10.x or higher, for example Python 3.12.4.

If it goes wrong:

- If it says *"Python was not found"* and opens the Microsoft Store, don't click anything in the Store. Just tell me, and I'll walk you through the install.
- If it shows a version below 3.10, tell me the number.

Reply with "done" and paste what the window printed.

Planning commits and pushes across project phases.

Planning commits and pushes across project phases.

The status is exactly what it should be. The three code files are staged as renames into Lab-4/, and both videos sit inside Lab-4/ waiting to be added. There's no `__pycache__`, and the root README.md and .gitignore are untouched.

Phase 7, Step 2: Commit and push to your own repo

With the prompt on `D:\SE\62_scramble>`, run these one at a time:

```
git add Lab-4
git commit -m "Lab-4: move project into Lab-4 and add before/after videos"
git push origin main
```

- `git add Lab-4` stages the two videos (the renames were already staged).
- `git push origin main` sends your 6 local commits to origin, which is your fork (inchara13kuppall/62_scramble). Nothing goes to SETAPESU26, and no pull request is created.

What you should see:

- After the commit: 5 files changed (3 renames and 2 videos).
- After the push: lines about "Enumerating / Writing objects", ending with `main -> main`.

If it goes wrong:

- If a browser window pops up asking you to sign in to GitHub, sign in as inchara13kuppall and approve. Then, if the terminal stopped, run `git push origin main` again.
- If you see rejected or 403, paste the message and I'll look. Don't try `--force`.

Once the push succeeds, check it on GitHub: open

https://github.com/inchara13kuppall/62_scramble and confirm you see a Lab-4 folder containing

main.py, game/, before_video.mp4 and after_video.mp4. If GitHub shows a yellow banner offering "Contribute" or "Open pull request", ignore it.

Reply "done" and paste the last lines of the push output, plus what you see on GitHub.

done, also tell what should be added in the doc and complete deliverable

Checking how to share and export the chat properly.

Phase 7 is almost done, and the push and the GitHub check are the last parts of it. Before I explain the export steps, I'm going to look up the current steps for sharing a chat, because that menu may have changed.

Searched the web

I'm taking "done" to mean your push went through. If you haven't opened

https://github.com/inchara13kuppal/62_scramble yet, do that once to check that the Lab-4 folder is there with the files listed below.

Phase 7 done, next: Phase 8 (chat PDF and share link)

What the complete deliverable looks like

Everything goes in Lab-4/ in your repo:

Item	File	Status
Video before changes	before_video.mp4	Pushed
Video after changes	after_video.mp4	Pushed
Updated code	main.py, game/game_engine.py, game/text_box.py	Pushed
Chat history	chat_history.pdf	To do
Chat share link	chat_link.txt (the URL inside, one line)	To do

The chat_link.txt file is optional, but it keeps the link in the repo next to everything else. The README's submission list asks for the two videos and the chat link, so submit the link wherever your course asks for it (I can't see your submission portal).

What goes in the doc

The PDF should be the complete, unedited chat from the first message to the last. Don't summarise it or cut anything out, because the chat history is what gets graded. You don't need to add any text of your own.

Do the export last

The share link is a snapshot. It includes every message sent before you share, while anything sent afterwards stays private, unless you unshare and share again, which refreshes the snapshot. So we'll run the final checklist first, and then you'll export and share, so the PDF and the link contain the whole chat. claude

Two other things to know:

- Attached files aren't included in a shared snapshot, only the conversation itself. Your screenshots therefore probably won't appear in the link, but they will in a PDF printed from the live chat. claude
- If you're on a Team or Enterprise plan, sharing is limited to people in your organization, not the public. Check that your link opens for someone outside your account, for example in a private browser window. claude

I'll walk you through the actual export and link steps after the checklist.

Next step: confirm GitHub

Open https://github.com/inchara13kuppai/62_scramble and click into the Lab-4 folder. It should contain main.py, game/, before_video.mp4 and after_video.mp4. Don't click any "Contribute" or "Open pull request" banner.